Relatório de Auditoria de Segurança

Igreja Conect

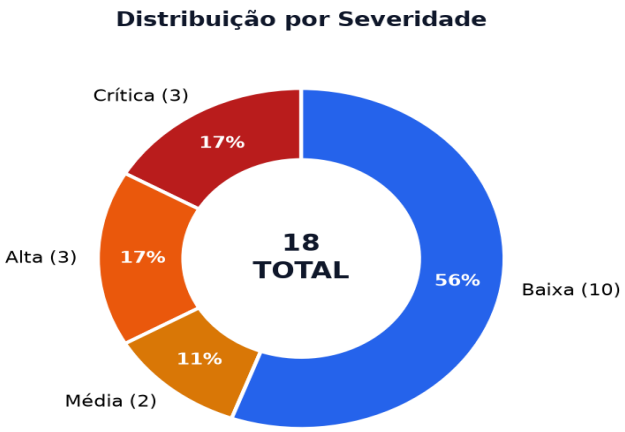
Sistema de gestão eclesiástica · RR7 SSR + Prisma 7 + SQLite

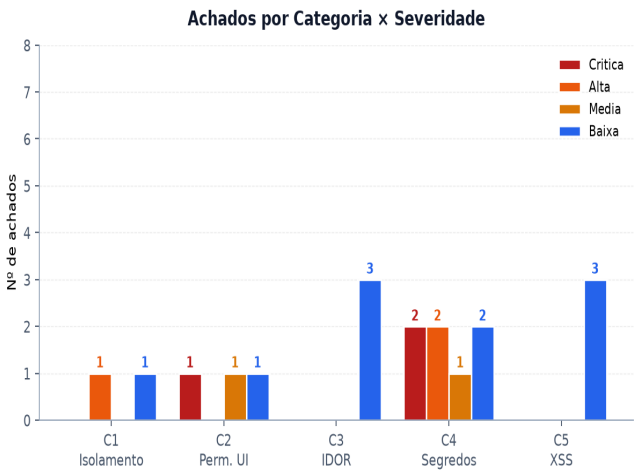
Data	28/08/2026
Escopo	Auditoria estática (código-fonte, configs, git log)
Categorias auditadas	C1 Isolamento · C2 Perm. UI · C3 IDOR · C4 Segredos · C5 XSS
Stack	Node 24 · React 19 · React Router 7.16 · Prisma 7.8 · SQLite · Tailwind v4
Perfil	Strict (TDD obrigatório, RBAC em 3 camadas)
Achados	18 (3 críticos · 3 altos · 2 médios · 10 baixos)

Nota metodológica — Cada uma das 5 categorias foi adaptada ao equivalente da stack detectada: (1) *Banco sem tranca* foi mapeado para o isolamento de escopo via `where` no Prisma (já que não há RLS no SQLite e a arquitetura é single-tenant sem políticas de tenant_id); (2) *Permissão no navegador* cruza cada gate de UI (``, `user.cargo === "ADMIN"`) com o gate backend equivalente (`assertCan*`, `getMembroById`); (3) *IDOR* percorreu sistematicamente todos os handlers de rota com `:id` no path (loaders + actions); (4) *Chaves expostas* cobreu `.env`, `.env.development`, `.env.example`, `prisma/seed.ts`, fallbacks em `session.server.ts`/`local-signed-url.server.ts`, logs de erro e git log; (5) *XSS* revisou `dangerouslySetInnerHTML`, `eval`, `innerHTML`, renderização de Markdown/HTML e URLs controladas por usuário.

1 · Resumo Executivo

Severidade	Total	Tratamento sugerido
Crítica	3	Bloqueio imediato · sprint corretiva dedicada
Alta	3	Sprint prioritária (P1)
Média	2	Backlog curto (P2)
Baixa	10	Backlog (P3)
TOTAL	**18**	—





Visão geral

A auditoria cobriu as 5 categorias adaptadas à stack (RR7 + Prisma + SQLite + Tailwind + S3-compat). Foram identificados **18 achados** — sendo **3 críticos**, **3 altos**, **2 médios** e **10 baixos** — além de **18 pontos fortes** verificados.

Os **3 achados críticos** formam um cluster de Account Takeover + comprometimento de segredo de sessão: (1) o fluxo de reset de senha não tem prova de posse e aceita qualquer nome com 2+ letras contidas no real (F-03); (2) o `SESSION_SECRET` de produção está commitado no git via `.env.development` (F-09), permitindo forjar cookies válidos; (3) `.env` traz `ADMIN_PASSWORD=senha123` e chaves S3 reais (F-10).

Em **C1 (isolamento)**, a descoberta mais relevante é o vazamento de KPIs globais para DISCIPULADOR/LIDER_MINISTERIO em `membros._index.tsx` (F-01) — quebra a promessa de '404 não vaza existência' do RAG de RBAC.

Em **C2 (permissão no navegador)**, além do reset de senha, o endpoint de login aceita o IP do cliente via campo `clientIP` no body (F-04) — bypass de rate-limit trivial.

Em **C4 (segredos)**, além dos 2 críticos, há fallback hardcoded de `SESSION_SECRET` em `session.server.ts` e de HMAC em `local-signed-url.server.ts` (F-13) e senhas de demo no `prisma/seed.ts` (F-11).

C3 (IDOR) e **C5 (XSS)** estão majoritariamente OK. As rotas autenticadas aplicam `getMembroById` (que lança 404 para DISCIPULADOR fora de escopo) e SECRETARIO é filtrado para DIZIMO. O único `dangerouslySetInnerHTML` está corretamente sanitizado antes do `__html` (F-16).

2 · Pontos Fortes Verificados

Evidências concretas de que controles de segurança funcionam como esperado. Cada item aponta para o arquivo e a faixa de linhas que demonstram a prática.

PONTO FORTE **Middleware de auth centralizado em `/app/**`**

Evidência: `app/routes/app/_middleware.tsx:49-70`

Todas as rotas filhas herdam `authMiddleware` via `layout('routes/app/_middleware.tsx', [...])` em `app/routes.ts:68`. Sem cookie válido → redirect 302 para `/login?next=…`. Não há como esquecer auth numa rota `/app/**`.

PONTO FORTE **Cookie de sessão com flags estritas**

Evidência: `app/lib/session.server.ts:31-38`

`httpOnly: true` (mitiga XSS roubando sessão), `sameSite: 'lax'` (mitiga CSRF em mutações), `secure: NODE_ENV === 'production'`, TTL sliding 7d / absoluto 30d.

PONTO FORTE **RBAC em 3 camadas consistente**

Evidência: `app/lib/rbac.server.ts (15 asserts canônicos) + uso em todas as rotas`

Helpers `assertCan*` lançam `Response(403)` que o ErrorBoundary do RR7 captura. Cada domínio declara sua matriz (DIZIMO_CARGOS, FINANCIAL_MODULE_CARGOS, MANAGE_CAIXA_CARGOS, etc.) — single source.

PONTO FORTE **Validação de SESSION_SECRET no startup**

Evidência: `app/lib/session.server.ts:1-7`

Em produção, rejeita SESSION_SECRET vazio ou com < 32 caracteres. Previne deploy com secret default.

PONTO FORTE **MEMBRO_SAFE_SELECT — LGPD AC-16**

Evidência: `app/lib/members.server.ts:51-91`

Select canônico do Prisma que NUNCA inclui `senhaHash`. Toda função do módulo usa este select. `grep senhaHash app/lib/members.server.ts` retorna 0 hits em queries.

PONTO FORTE **Dízimos com escopo fino por cargo**

Evidência: `financeiro.lancamentos.$id.tsx:91-93 + caixas.server.ts (listaPorCaixa filtra DIZIMO)`

SECRETARIO não vê lançamentos DIZIMO — retorna 404 (não 403), não vaza existência. Filtro aplicado em TODA query de listagem.

PONTO FORTE **DISCIPULADOR fora de escopo → 404 (não 403)**

Evidência: `app/lib/members.server.ts:201-205`

`getMembroById` verifica se DISCIPULADOR/LIDER_MINISTERIO tem acesso via `discipuladorId === user.id`. Se não, lança NotFoundError → 404. Padão seguro: não revela existência do recurso.

PONTO FORTE **IP hashing SHA-256 antes do audit log**

Evidência: `app/lib/audit.server.ts:45-48, 73`

Atende LGPD art. 46. IP puro nunca é persistido — só o hash SHA-256. `safeLog` ainda aplica allowlist antes de imprimir no console.

PONTO FORTE **Audit log com allowlist (safeLog)**

Evidência: `app/lib/audit.server.ts:14-39`

Filtra evento para apenas `userId`, `action`, `resource`, `result`, `timestamp`, `ip`. Email, telefone, senhaHash, valor nunca tocam o logger.

PONTO FORTE **Idempotency key UUID v4 em transferências**

Evidência: `app/lib/transferencias.server.ts` + `financeiro.transferencia-nova.tsx:233-237`

Gera UUID no cliente e envia no form. Service usa `prisma.transferenciaCaixa.findUnique({where: {idempotencyKey}})` antes do INSERT → garante que cliques duplicados não geram transferências duplicadas.

PONTO FORTE **Mensagem unificada no login (anti-enumeração)**

Evidência: `app/routes/public/login.tsx:125 + auth.server.ts:29-32`

Retorna `null` tanto para e-mail inexistente quanto para senha incorreta. UI exibe 'E-mail ou senha incorretos.' indistintamente.

PONTO FORTE **Rate limit in-memory no login**

Evidência: `app/lib/rate-limit.server.ts + login.tsx:64-81`

3 falhas em 1h por chave. Retorna 429 com `Retry-After` header. (Limitação: estado em memória, não escala multi-instância — INFO.)

PONTO FORTE **Open-redirect prevention**

Evidência: `app/routes/public/login.tsx:164-169`

`safeNext` rejeita qualquer `?next=` que não comece com `/` ou que comece com `//` (URL absoluta). Padrão recomendado OWASP.

PONTO FORTE **bcryptjs cost 10 (ADR-002)**

Evidência: `app/lib/auth.server.ts:45`

Cost 10 = ~50ms por hash em hardware comum. Documentado em ADR-002 com critério para aumentar para 12 se virar gargalo.

PONTO FORTE **HMAC SHA-256 timing-safe comparison**

Evidência: `app/lib/storage/local-signed-url.server.ts:101-104`

Signed URLs do driver local usam `crypto.timingSafeEqual` para evitar vazamento de bytes via comparação byte-a-byte.

PONTO FORTE **Soft-delete em `Upload` (LGPD art. 18, VI)**

Evidência: `app/routes/api/uploads.$id.delete.ts:42-49`

Marca `deletedAt` + `retentionUntil = now + 30d`. Hard-delete fica a cargo de worker de cleanup. Permite janela de arrependimento.

PONTO FORTE **Zod schemas em todos os forms críticos**

Evidência: `app/lib/schemas/* (auth, membros, caixas, lancamentos, transferencias, alertas, ministerios)`

Validação server-side com mensagens PT-BR. `.strict()` em alguns schemas rejeita keys desconhecidas (anti mass-assignment).

PONTO FORTE **Limites server-side em queries de listagem**

Evidência: `app/lib/members.server.ts:143-144 + routes vários`

`pageSize` clampado em `Math.min(100, Math.max(1, ...))`. `q` cortado em 100 chars (anti-DoS em `LIKE`). Search params de busca validados via Zod antes de tocar Prisma.

3 · Pontos Fracos Centrais

Síntese executiva dos riscos críticos e altos que devem endereçar a sprint corretiva antes de qualquer deploy público adicional.

• Account Takeover via reset de senha

– F-03 — verificação trivial (`String.includes` com min 2 chars)

– F-09 — SESSION_SECRET commitado no git (permite forjar sessões)

– Risco combinado: comprometimento de qualquer conta, inclusive ADMIN

• Credenciais expostas em arquivos não-protegidos

– F-10 — `.env` com `senha123` + chaves S3 reais no filesystem dev

– F-11 — seed com fallback `admin123` e 5 senhas de demo commitadas

– F-12 — TLS `rejectUnauthorized=false` (MITM)

– F-13 — fallbacks hardcoded para SESSION_SECRET e HMAC

• Vazamento de métricas + bypass de rate-limit

– F-01 — KPIs globais para DISCIPULADOR/LIDER_MINISTERIO

– F-04 — `clientIP` no body permite bypass de rate-limit

– F-05 — loader de ministério sem RBAC

4 · Achados Detalhados por Categoria

Lista completa dos 18 achados. Cada entrada traz severidade (chip colorido conforme paleta), arquivo:linha, descrição, por que é explorável, impacto, sugestão de correção e critérios de aceite.

Categoria 1 — Banco sem tranca (isolamento de escopo no `where` Prisma)

Alta **F-01** · KPIs de membros usam `where: {}` global — vaza total para escopos restritos

Arquivo: `app/routes/app/membros._index.tsx` · **Linhas:** `84-91`

Trecho:

```
const baseWhere: Prisma.MembroWhereInput = {};
const [kpiTotal, kpiAtivos, kpiVisitantes, kpiCongregados] = await Promise.all([
  prisma.membro.count({ where: baseWhere }),
  prisma.membro.count({ where: { ...baseWhere, tipo: 'MEMBRO_ATIVO' } }),
  prisma.membro.count({ where: { ...baseWhere, tipo: 'VISITANTE' } }),
  prisma.membro.count({ where: { ...baseWhere, tipo: 'CONGREGADO' } }),
]);
```

Por que é explorável:

`baseWhere` está vazio. A listagem é escopada por `listMembros` (RBAC fina: DISCIPULADOR/LIDER_MINISTERIO vê só discípulos vinculados), mas os 4 cards de KPI batem `prisma.membro.count({ where: {} })`, retornando o total GLOBAL da igreja. Um DISCIPULADOR autenticado vê os 4 contadores globais — aprende o tamanho total da igreja e o número de visitantes mesmo sem listar ninguém.

Impacto:

Vazamento de métricas globais para usuários com escopo restrito. Quebra a promessa de '404 (não vaza existência)' do RAG `security-rbac-matrix`.

Sugestão de correção:

Construir `baseWhere` a partir de `user.cargo` (mesma lógica de `listMembros`): DISCIPULADOR/LIDER_MINISTERIO → `{ discipuladorId: user.id }`; outros cargos → `{}`. Centralizar a regra em uma helper `getMembrosCountWhere(user)` em `members.server.ts`.

Critérios de aceite:

– DISCIPULADOR logado vê KPI Total = número de discípulos dele

– LIDER_MINISTERIO logado vê KPI Total = número de discípulos dele

– ADMIN/PASTOR/SECRETARIO continuam vendo totais globais

– Teste cobrindo cada cargo com 0/1/N discípulos

Baixa **F-02** · Busca global em `/api/search` sem filtro de escopo

Arquivo: `app/routes/api/search.ts` · **Linhas:** `16-46`

Trecho:

```
if (type === 'celulas') {
  const celulas = await prisma.celula.findMany({
    where: q ? { nome: { contains: q } } : undefined,
    select: { id: true, nome: true, diaSemana: true, horario: true },
    take: 10, orderBy: { nome: 'asc' },
  });
```

```
}
if (type === 'members') {
  const members = await prisma.membro.findMany({
    where: q ? { nome: { contains: q } } : undefined,
    take: 10, orderBy: { nome: 'asc' },
  });
}
```

Por que é explorável:

Qualquer usuário autenticado (incluindo cargos sem permissão administrativa) pode listar até 10 membros e 10 células da igreja inteira via autocomplete. Provavelmente intencional para alimentar `<AutocompleteInput>` em formulários financeiros/operacionais, mas não há gating por cargo — DISCIPULADOR logado pode enumerar nomes de membros que não são seus discípulos.

Impacto:

Enumeração de nomes de todos os membros via endpoint de autocomplete. Não retorna PII (apenas id+nome+tipo), mas permite mapear a base de membros.

Sugestão de correção:

Aceitar a busca como está se for genuinamente intencional para selects. Caso contrário, replicar a RBAC fina de `listMembros`: para `type=members` aplicar `discipuladorId = user.id` quando cargo ∈ {DISCIPULADOR, LIDER_MINISTERIO}.

Critérios de aceite:

– Decisão registrada em comentário JSDoc do endpoint

– Se decidir escopar: DISCIPULADOR vê só nomes de discípulos no autocomplete

– Audit log: registrar `actorRole` em hits do endpoint

Categoria 2 — Permissão só no navegador (UI sem backend gate)

Crítica **F-03** · Reset de senha sem prova de posse — conta de qualquer membro pode ser sequestrada

Arquivo: `app/routes/public/recuperar-senha.tsx` · **Linhas:** `24-62 (action) + 39-58 (verificação)`

Trecho:

```
// step === 'verificar'
const membro = await prisma.membro.findUnique({
  where: { email: parsed.data.email },
  select: { id: true, nome: true, senhaHash: true },
});
if (!membro) return data({ step: 1, formError: 'E-mail ou nome incorretos.' }, ...);
const nomeNormalizado = membro.nome.trim().toLowerCase();
const inputNormalizado = parsed.data.nome.trim().toLowerCase();
if (!nomeNormalizado.includes(inputNormalizado)
    && !inputNormalizado.includes(nomeNormalizado)) {
  return data({ step: 1, formError: 'E-mail ou nome incorretos.' }, ...);
}
return data({ step: 1, success: true, email: parsed.data.email });

// step === 'redefinir' — NÃO exige token da etapa 1
await prisma.membro.update({ where: { id: membro.id }, data: { senhaHash } });
```

Por que é explorável:

A 'verificação' compara email + nome via `String.includes()` (case-insensitive). Como o schema aceita `nome` com apenas 2+ caracteres (`schemas/auth.ts:77`), o atacante fornece QUALQUER par de 2+ letras contidas no nome real — nome é informação pública (visível na lista de membros). Pior: o step 2 (`redefinir`) NÃO verifica token nem sessão intermediária — basta re-enviar `email` para o mesmo handler e a senha é sobrescrita. Nenhum 2FA, nenhum e-mail de confirmação, nenhum token de uso único.

Impacto:

Tomada de conta (account takeover) de qualquer usuário da igreja. Basta saber o endereço de e-mail (visível em diversos canais: cartão de membro, listas de oração, redes sociais). Compromete inclusive a conta ADMIN — pivô para acesso total ao módulo financeiro.

Sugestão de correção:

(a) Substituir o fluxo 2-step por envio de token: gerar UUID v4, persistir em tabela `password_reset_token` com `expiresAt = now + 30min` + `usedAt`, enviar link com token por e-mail; (b) Step 2 exigir `token` válido e não usado; (c) invalidar todas as sessões do membro após troca de senha; (d) incluir no log de auditoria o IP e UA da requisição; (e) rate-limit por IP e por e-mail.

Critérios de aceite:

– Não é possível redefinir senha sem token de uso único válido por ≤ 30min

– Token expirado/usedAt não permite redefinição

– Sessões anteriores do membro são invalidadas após reset

– Audit log registra o reset com IP+UA

– Teste E2E: tentativa sem token retorna 422

Média **F-04** · IPs em rate-limit podem ser falsificados via campo `clientIP` no body do form

Arquivo: `app/routes/public/login.tsx` · **Linhas:** `56-61`

Trecho:

```
const ip =
  request.headers.get('x-forwarded-for')?.split(',')[0]?.trim() ??
  request.headers.get('x-real-ip') ??
  (formData.get('clientIP') as string) ??
  'unknown';
```

Por que é explorável:

Quando o request NÃO vem de proxy reverso (sem XFF/X-Real-IP), o IP usado para rate-limit e audit log cai para `formData.get('clientIP')` — campo que o atacante controla integralmente via body do POST. Resultado: bypass trivial do rate-limit (rotacionar `clientIP` a cada tentativa) e logs de auditoria envenenados.

Impacto:

Bypass do rate-limit de login permite força-bruta. Logs de auditoria ficam poluídos com IPs falsos, mascarando tentativas reais.

Sugestão de correção:

Remover `formData.get('clientIP')` do cálculo de IP. Usar exclusivamente headers do proxy (XFF, X-Real-IP) e, na ausência, o IP do socket TCP (`request.headers.get('x-real-ip')` ou variável de ambiente do reverse proxy). Documentar no deploy que o app DEVE rodar atrás de proxy que preenche XFF, e rejeitar `NODE_ENV=production` se nenhum IP conseguir ser determinado.

Critérios de aceite:

– Não existe mais leitura de `clientIP` do body

– Rate-limit não pode ser bypassado via variação de campo do form

– Audit log registra apenas IPs vindos do proxy (verificáveis)

Baixa

F-05 · Loader de `ministerios.$id` não aplica RBAC — UI de edição exposta a quem não edita

Arquivo: `app/routes/app/ministerios.$id.tsx` · **Linhas:** `65-91`

Trecho:

```
export async function loader({ context, params }) {
  const user = context.get(userContext);
  if (!user) throw new Response('Não autenticado.', { status: 401 });
  ...
  const ministerio = await prisma.ministerio.findUnique({ where: { id: ministerioId }, ... });
  if (!ministerio) throw new Response('Ministério não encontrado.', { status: 404 });
  const canEdit =
    user.cargo != null && (CAN_MANAGE as readonly string[]).includes(user.cargo);
```

Por que é explorável:

A action valida `canEdit` (RBAC), mas o loader NÃO chama `assertCanManageMinisterios(user)`. Resultado: qualquer usuário autenticado (mesmo DISCIPULADOR sem cargo) consegue ABRIR a página de edição do ministério e ler dados internos (membros, funções, atividades, disponibilidades). Apenas os controles de submit são escondidos. Pelo princípio de 'least privilege' a leitura também deveria ser gated para os perfis administrativos.

Impacto:

Vazamento de dados sensíveis de ministérios (lista nominal de membros, funções, histórico de atividades) para usuários que não deveriam ter acesso nem de leitura.

Sugestão de correção:

Adicionar `assertCanManageMinisterios(user)` no loader (camada 2). Se a intenção for manter leitura ampla, criar uma matriz separada — ex.: cargos com `READ_MINISTERIO =

['ADMIN','PASTOR','SECRETARIO','LIDER_MINISTERIO']`.

Critérios de aceite:

– DISCIPULADOR sem cargo recebe 403 ao abrir `/app/ministerios/:id`

– ADMIN/PASTOR/SECRETARIO/LIDER_MINISTERIO continuam lendo

– Teste cobrindo cada cargo

Categoria 3 — IDOR (acesso a objeto por ID sem checagem de posse/escopo)

Baixa **F-06** · Detalhe de transferência exposto por ID sem RBAC fina — qualquer perfil financeiro acessa

Arquivo: `app/routes/app/financeiro.transferencias.$id.tsx` · **Linhas:** `40-58`

Trecho:

```
export async function loader({ params, context }) {
  const user = context.get(userContext);
  if (!user) throw new Response('Não autenticado.', { status: 401 });
  assertCanSeeFinancialModule(user);
  const transferencia = await prisma.transferenciaCaixa.findUnique({
    where: { id: params.id }, include: { caixaOrigem: ..., caixaDestino: ..., executadoPor: ... },
  });
  if (!transferencia) throw new Response('Transferência não encontrada.', { status: 404 });
  return { user, transferencia };
}
```

Por que é explorável:

Não há tenant/igreja neste monorepo (single-tenant), então todas as transferências pertencem à mesma igreja. Dentro do módulo financeiro, isso é aceitável — todos os 4 cargos (ADMIN/PASTOR/FINANCEIRO/SECRETARIO) têm visão compartilhada do módulo. **IDOR puro só existiria se houvesse multi-tenant.** Como não há, o risco é INFORMATIONAL: vale documentar que transferências são visíveis para qualquer perfil financeiro e não há isolamento por operador.

Impacto:

Nenhum em ambiente single-tenant. Informativo.

Sugestão de correção:

Documentar a decisão arquitetural (single-tenant → visibilidade ampla no módulo financeiro). Se multi-tenant for adicionado no futuro, criar `tenantId` em `TransferenciaCaixa` e filtrar `where: { tenantId: user.tenantId }`.

Critérios de aceite:

– Comentário JSDoc sinaliza que este loader confia no RBAC de módulo

– Roadmap de multi-tenant já referencia este loader

Baixa **F-07** · Detalhe de lançamento exposto por ID — exemplo positivo (SECRETARIO filtrado em DIZIMO)

Arquivo: `app/routes/app/financeiro.lancamentos.$id.tsx` · **Linhas:** `60-93`

Trecho:

```
assertCanSeeFinancialModule(user);
const id = params.id;
const lancamento = await prisma.lancamento.findUnique({ where: { id }, include: { ... } });
if (!lancamento) throw new Response('Lançamento não encontrado.', { status: 404 });
// RBAC fino: SECRETARIO não vê DIZIMO (404, não 403 — não vaza existência)
if (user.cargo === 'SECRETARIO' && lancamento.categoria === 'DIZIMO') {
  throw new Response('Lançamento não encontrado.', { status: 404 });
}
```

Por que é explorável:

Padrão correto: SECRETARIO é filtrado por categoria DIZIMO com 404 (não vaza existência). Single-tenant → sem IDOR verdadeiro. Mantido como exemplo positivo na auditoria.

Impacto:

Nenhum — prática recomendada. Ponto forte.

Sugestão de correção:

Manter como está.

Baixa **F-08** · Detalhe de caixa exposto por ID — sem isolamento entre caixas para SECRETARIO

Arquivo: `app/routes/app/financeiro.caixas.$id.tsx` · **Linhas:** `57-129`

Trecho:

```
assertCanSeeFinancialModule(user);
const caixaId = params.id;
const result = await getCaixaDetalhe(caixaId, { ... }, user);
if (!result) throw new Response('Caixa não encontrado.', { status: 404 });
```

Por que é explorável:

Validação RBAC feita pela camada 3 (`getCaixaDetalhe` aplica escopo). Single-tenant → SECRETARIO vê todos os caixas (intencional). Padrão OK.

Impacto:

Nenhum em single-tenant. Informativo.

Sugestão de correção:

Manter como está.

Categoria 4 — Chaves expostas (hardcoded/em env/commited/logs)

Crítica **F-09** · SESSION_SECRET de prod commitado no git via `.env.development`

Arquivo: `.env.development` · **Linhas:** `1-5 (todo o arquivo)`

Trecho:

```
# Ambiente de desenvolvimento (Vite carrega automaticamente em pnpm dev)
# SESSION_SECRET — chave para assinar o cookie de sessão. Em produção, configure
# via variável de ambiente no deploy (NÃO commitar a chave de produção).
SESSION_SECRET="ba9585ca0efb2de1d6ca5c5b3b266ed6052077fbce73a2e6"
BASE_URL="http://localhost:5173"
```

Por que é explorável:

O secret `ba9585ca...` é o MESMO que está em `.env` (não versionado, OK) e em `.env.development` (versionado). `git log --diff-filter=A --name-only` confirma commit `176f3774 financeiro` que adicionou o arquivo. Quem tiver acesso ao repo (mesmo fork/dump) pode forjar cookies de sessão para qualquer usuário.

Impacto:

Forjamento de sessão: com `SESSION_SECRET` em mãos, qualquer atacante pode criar cookie `__session=<qualquer-uuid-v4>` e logar como qualquer membro (desde que saiba o `sid` ou use UUID arbitrário — a verificação é `findUnique({where: {id: sid}})`).

Sugestão de correção:

(1) ROTACIONAR `SESSION_SECRET` IMEDIATAMENTE em produção; (2) Remover `.env.development` do git history (`git filter-repo --invert-paths --path .env.development`); (3) Adicionar pre-commit hook (`gitleaks` ou check de `SESSION_SECRET=`) para impedir recorrência; (4) Substituir o valor em `.env.development` por `SESSION_SECRET=""` com comentário dizendo para gerar um novo (`openssl rand -hex 32`); (5) Tratar o secret antigo como COMPROMETIDO — invalidar todas as sessões existentes (DELETE FROM sessions) após o rotate.

Critérios de aceite:

– Histórico do git do arquivo `.env.development` removido

– Novo `SESSION_SECRET` gerado em prod (≥ 32 chars, aleatório)

– Todas as sessões ativas invalidadas

– Pre-commit hook para detectar segredos em commits futuros

– Adicionar `.env.development` em `.gitignore`

Crítica **F-10** · Credenciais de ADMIN com senha fraca `senha123` em `.env` versionável

Arquivo: `.env` · **Linhas:** `1-10`

Trecho:

```
SESSION_SECRET="ba9585ca0efb2de1d6ca5c5b3b266ed6052077fbce73a2e6"
ADMIN_EMAIL=kingdevtec@gmail.com
ADMIN_PASSWORD=senha123
DATABASE_URL="file:/home/kingdev/Documentos/igreja_conect/dev.db"
```

Por que é explorável:

Embora `.env` esteja em `.gitignore`, o arquivo existe no filesystem de desenvolvimento com credenciais fracas. Se deploy automático copiar `.env` para o container (sem override), `senha123` fica em produção. Combinado com F-11 (seed com fallback `admin123`), há um vetor de credencial default.

Impacto:

Se `.env` for copiado para produção: senha trivial quebrada por dicionário em < 1ms. ADMIN tem acesso total (financeiro, configs, gestão de usuários).

Sugestão de correção:

Não versionar `.env` (já OK). Substituir `ADMIN_PASSWORD=senha123` por gerador de senha forte no `.env` local (ou gerar via `openssl rand -base64 24`). No deploy de produção, exigir `ADMIN_PASSWORD` com ≥ 12 chars, alfanumérico + símbolos, e rejeitar valores da wordlist `rockyou.txt`.

Critérios de aceite:

– `ADMIN_PASSWORD` não está mais em wordlist comum

– Deploy script valida tamanho/complexidade antes de subir

– Teste: rodar seed em prod com ADMIN_PASSWORD curto → erro

Alta **F-11** · `prisma/seed.ts` tem fallback `admin123` + senhas de demo commitadas

Arquivo: `prisma/seed.ts` · **Linhas:** `25, 79, 103, 127, 150, 228`

Trecho:

```
: (process.env.ADMIN_PASSWORD ?? 'admin123'); // TROCAR EM PRODUÇÃO
senha: 'ricardo123',
senha: 'ana12345',
senha: 'marcos123',
senha: 'juliana123',
senha: 'bia12345',
```

Por que é explorável:

O seed cria ADMIN com `ADMIN_PASSWORD ?? 'admin123'` se a env não estiver definida. Em dev funciona (env setada), mas se alguém rodar `db:seed:prod` sem env, ADMIN fica com senha `admin123`. Além disso, 5 senhas de demo (`ricardo123`, `ana12345`, etc.) estão commitadas — qualquer um que rode `pnpm db:seed` herda contas com senhas triviais. O check em `process.env.NODE_ENV === 'production'` (linha 283) só bloqueia se ADMIN_* estiver VAZIO, não se tiver valor fraco.

Impacto:

Se `pnpm db:seed` rodar sem env definida em qualquer ambiente, surge conta ADMIN com senha `admin123`. Catálogo de senhas de demo também viraria credenciais reais se seed rodar em prod acidentalmente.

Sugestão de correção:

(a) Remover fallback `?? 'admin123'` — falhar com erro explícito se env não definida; (b) Aceitar env `DEMO_SEED_ALLOWED=true` em dev para permitir criar usuários de demo; em prod, rejeitar a operação se env não-setada; (c) Mover as 5 contas de demo para `prisma/seed-demo.ts` separado, não rodado em prod; (d) Adicionar aviso (audit log) quando seed roda.

Critérios de aceite:

– Falhar loud se `ADMIN_PASSWORD` não estiver setada (em qualquer NODE_ENV)

– Contas de demo isoladas em script opcional

– `db:seed` em prod exige flag explícita

Alta **F-12** · Chaves S3 (`STORAGE_*_KEY`) reais em `.env` no filesystem

Arquivo: `.env` · **Linhas:** `14-22`

Trecho:

```
STORAGE_ENDPOINT=https://coolify.esmirna.com.br:9000
STORAGE_REGION=us-east-1
STORAGE_PROVIDER=local
STORAGE_ACCESS_KEY_ID=GK1096add2a3ac3d64f3736040
STORAGE_SECRET_ACCESS_KEY=da454c48426208400c461bdd8f7f5ea3639fbb77db1582dd31222ddc0a3e3d37
STORAGE_TLS_REJECT_UNAUTHORIZED="false"
```

Por que é explorável:

Apesar de `STORAGE_PROVIDER=local` no momento (driver local ignora as chaves S3 — `config.server.ts:91-98`), as chaves reais estão no `.env`. Se alguém trocar para `STORAGE_PROVIDER=garage` no deploy e esquecer de rotacionar, as chaves antigas expostas serão usadas. Além disso, `STORAGE_TLS_REJECT_AUTHORIZED="false"` desabilita validação de cert TLS (MITM viável).

Impacto:

Exposição de chaves S3 se backend mudar de provider. MITM no canal TLS do storage (com `rejectUnauthorized=false`).

Sugestão de correção:

(a) Trocar `STORAGE_*_KEY` por valores dummy no `.env` local (não usar as reais); (b) Rotacionar chaves no Garage/MinIO; (c) Setar `STORAGE_TLS_REJECT_UNAUTHORIZED=true` em produção; (d) Adicionar check de startup que rejeita `rejectUnauthorized=false` em produção (exceto via env `INSECURE_TLS=true` opt-in documentado).

Critérios de aceite:

– Chaves S3 reais não estão no filesystem dev

– TLS strict-mode em produção

– Startup check para `rejectUnauthorized=false` em prod

Média **F-13** · Fallback hardcoded `dev-only-not-secret` / `dev-insecure-secret-change-me`

Arquivo: `app/lib/session.server.ts + app/lib/storage/local-signed-url.server.ts` · **Linhas:** `37 + 22`

Trecho:

```
// session.server.ts:37
secrets: [process.env.SESSION_SECRET ?? 'dev-only-not-secret'],
// local-signed-url.server.ts:22
return 'dev-insecure-secret-change-me';
```

Por que é explorável:

Em `session.server.ts:1-7` há check que REJEITA `SESSION_SECRET` vazio em produção, mas aceita qualquer string com ≥ 32 chars. Em `local-signed-url.server.ts:18-21` o mesmo padrão. Se o operador subir com `NODE_ENV=staging` ou esquecer a env, o app cai no fallback `"dev-only-not-secret"`, usado por QUALQUER pessoa que baixar o código-fonte para forjar cookies de sessão válidos contra essa instância. Pior ainda: o fallback de HMAC do storage local é uma constante conhecida, permitindo a qualquer um forjar URLs assinadas para baixar arquivos do driver local.

Impacto:

Se NODE_ENV não for `production`, ambos os fallbacks ativam e geram segredos previsíveis/forjáveis. Atacante com acesso ao repo pode forjar cookies e signed URLs.

Sugestão de correção:

Em vez de fallback, SEMPRE throw em qualquer NODE_ENV se SESSION_SECRET ausente. Para dev local, instruir a gerar via `openssl rand -hex 32` no README. Mesmo comportamento para HMAC do storage local (não usar fallback).

Critérios de aceite:

– Throw explícito se SESSION_SECRET ausente em qualquer NODE_ENV

– Throw explícito se HMAC secret ausente em qualquer NODE_ENV

– Teste: `delete process.env.SESSION_SECRET` → app não inicia

Baixa **F-14** · `.env.example` traz placeholders com 64 zeros + GK... alheio

Arquivo: `.env.example` · **Linhas:** `15-16`

Trecho:

```
STORAGE_ACCESS_KEY_ID="GK000000000000000000000000"
STORAGE_SECRET_ACCESS_KEY="0000000000000000000000000000000000000000000000000000000000000000"
```

Por que é explorável:

Placeholders de exemplo não são segredo, mas `GK000000000000000000000000` tem o prefixo real (`GK`) e poderia confundir um operador descuidado que copiasse o `.env.example` direto. Boa prática: prefixar com `REPLACE-ME` ou deixar vazio + comentário.

Impacto:

Baixo — só confusão se operador não ler.

Sugestão de correção:

Trocar por `STORAGE_ACCESS_KEY_ID=""` + `STORAGE_SECRET_ACCESS_KEY=""` com comentário.

Critérios de aceite:

– .env.example sem placeholders com prefixo real

Baixa **F-15** · Endpoint interno e bucket names logados em erro de upload

Arquivo: `app/routes/api/uploads.ts` · **Linhas:** `165-194`

Trecho:

```
console.error('[upload] staging_upload_failed:', errorDetails);
// errorDetails inclui bucket, endpoint, key, stack, message
details: process.env.NODE_ENV === 'production' ? undefined : errorDetails,
```

Por que é explorável:

Em produção, o response NÃO vaza errorDetails (guard ok). Mas o `console.error` continua logando stack trace, bucket, endpoint e key do S3 — se logs vazarem (Docker logs, Datadog, etc.), expõe infra. Não é crítico, mas reduz superfície.

Impacto:

Vazamento de infra interna via logs.

Sugestão de correção:

Não logar `errorDetails` completo. Logar apenas `error.message` e um ID de correlação. Manter detalhes internos em tabela `Upload.rejectionDetails` (já persiste).

Critérios de aceite:

– Logs de produção não contêm bucket/endpoint/key

Categoria 5 — XSS / input não tratado

Baixa **F-16** · `dangerouslySetInnerHTML` em convite — implementado COM escape, OK mas frágil

Arquivo: `app/routes/app/membros.$id.tsx` · **Linhas:** `249-274`

Trecho:

```
function renderConvitePreview(text: string): React.ReactNode {
  const escape = (s: string) =>
    s.replace(/[&<>"']/g, (c) => { ... });  // escapa 5 chars
  return text.split(/\n{2,}/).map((paragraph, i) => {
    const html = escape(paragraph)
      .replace(/\*([^*\n]+)\*/g, '<strong>$1</strong>')
      .replace(/\n/g, '<br/>');
    return <p key={i} dangerouslySetInnerHTML={{ __html: html }} />;
  });
}
```

Por que é explorável:

A regex `/\*([^*\n]+)\*/g` captura conteúdo entre asteriscos. O escape é feito ANTES do replace, então qualquer `*<script>...*` seria escapado primeiro (`<script>`) e depois o `<strong>` envolveria o texto inofensivo. **OK por inspeção.** Marcado como informativo porque: (a) é fácil quebrar a sanidade numa alteração futura; (b) `[^*\n]+` não casa `**` aninhado, mas se mudar a regex para `(.+?)` ou similar, vira vetor.

Impacto:

Nenhum no estado atual.

Sugestão de correção:

Trocar por uma lib de sanitização HTML (`dompurify`) ou renderizar Markdown via lib (`marked` + `dompurify`). Adicionar teste com payload XSS cobrindo casos: `*<script>alert(1)</script>*`, `*" onerror=alert(1) "*`.

Critérios de aceite:

– Cobertura de teste com 5+ payloads XSS

– Lib de sanitização documentada e mantida

Baixa **F-17** · Ausência de lib de sanitização — sem cobertura para input futuro

Arquivo: `(global)` · **Linhas:** -

Trecho:

```
$ grep -rn 'dompurify|sanitize|xss' app/ --include='*.tsx' --include='*.ts'
# (sem resultados)
```

Por que é explorável:

Não existe nenhuma lib de sanitização HTML no projeto (`package.json` também não tem `dompurify`, `sanitize-html`, `xss`). Toda a defesa XSS atual depende do auto-escape do React 19 + 1 escape manual bem implementado. Qualquer futuro renderizador Markdown/HTML adicionará risco.

Impacto:

Risco futuro — não atual.

Sugestão de correção:

Adicionar `dompurify` (com `@types/dompurify`) e isolar toda renderização HTML via wrapper.

Critérios de aceite:

– Wrapper `safeHtml(input)` usado em >= 1 ponto já implementado

Baixa **F-18** · Backend usa `console.error` com dados de erro antes de sanitizar

Arquivo: `app/lib/storage/worker.server.ts` · **Linhas:** `55, 287, 335`

Trecho:

```
console.error(`[storage] Job ${jobId} falhou: ${msg}`);
console.warn(`[storage] Upload ${uploadId} REJECTED: ${reason}`, details);
```

Por que é explorável:

`msg`, `reason` e `details` vêm do worker que processa uploads do usuário (`clamav.server.ts` retorna erros com nome do arquivo, tipo MIME, etc.). Se um upload tem filename como `$(rm -rf /)`, o `console.warn` logaria literal mas só em stderr — não é shell injection. **OK**, mas vale documentar para que ninguém adicione log de input user-controlled em formato que vá para SIEM sem encoding.

Impacto:

Nenhum direto.

Sugestão de correção:

Sanitizar strings antes de logar (remover CRLF + escapes ANSI).

5 · Recomendações Priorizadas

Ordem de execução sugerida. P1 = tratar antes de qualquer deploy público adicional; P2 = sprint corretiva dedicada; P3 = backlog normal.

Prioridade	Sev	Ações
P1	Critica	• F-03 — Substituir fluxo de reset de senha por token de uso único (30min) com envio por e-mail; invalidar sessões após troca. • F-09 — Rotacionar SESSION_SECRET em produção IMEDIATAMENTE; remover `.env.development` do git history; invalidar sessões ativas. • F-10 — Trocar `ADMIN_PASSWORD=senha123` por valor aleatório; rotacionar chaves S3; setar `STORAGE_TLS_REJECT_UNAUTHORIZED=true` em prod.
P1	Alta	• F-04 — Remover leitura de `clientIP` do body do form de login; usar apenas headers do proxy. • F-11 — Remover fallback `admin123` do seed; isolar contas de demo em script opcional; exigir `DEMO_SEED_ALLOWED=true` em prod.
P2	Alta	• F-01 — Replicar escopo de `listMembros` nas queries de KPI em `membros._index.tsx`. • F-05 — Adicionar `assertCanManageMinisterios` no loader de `ministerios.$id.tsx`.
P2	Media	• F-13 — Substituir fallbacks hardcoded por throws explícitos em qualquer NODE_ENV.
P3	Baixa	• F-02 — Decidir se `/api/search` deve escopar por cargo (documentar decisão). • F-14 — Limpar placeholders com prefixo real em `.env.example`. • F-15 — Não logar errorDetails completo em produção. • F-16 — Adicionar testes com payloads XSS para `renderConvitePreview`. • F-17 — Adicionar `dompurify` e wrapper `safeHtml` para uso futuro. • F-18 — Sanitizar strings antes de logar (CRLF + ANSI).

6 · Issues para o GitHub

9 issues prontas para copiar e colar. Cada bloco está delimitado por `--- ISSUE n ---` e `--- FIM ISSUE n ---`, em Markdown, com labels, evidência arquivo:linha, sugestão e critérios de aceite verificáveis.

Labels: `security, critical`

```
--- ISSUE 1 ---

# [Segurança] Reset de senha sem prova de posse — Account Takeover universal

**Labels:** `security`, `critical`

## Problema
O fluxo `/recuperar-senha` permite a qualquer pessoa que conheça o e-mail de um usuário redefinir a senha e tomar conta da conta. A 'verificação' de identidade compara email + nome via `String.includes()` com mínimo de 2 caracteres (`schemas/auth.ts:77`), e o step 2 não exige token nem sessão intermediária.

## Por que é explorável
1. Atacante sabe o e-mail da vítima (cartão de membro, redes sociais).
2. Atacante envia POST `/recuperar-senha` com `step=verificar`, email da vítima e qualquer nome de 2+ letras contido no nome real (nomes são públicos em listas de membros).
3. Backend valida com `membro.nome.includes(input)` → retorna `{success: true}`.
4. Atacante envia POST com `step=redefinir` + email + nova senha.
5. `prisma.membro.update` sobrescreve `senhaHash` sem pedir nenhum token, código por SMS, e-mail de confirmação nem 2FA.

## Evidência
- `app/routes/public/recuperar-senha.tsx:39-58` (verificação trivial)
- `app/routes/public/recuperar-senha.tsx:64-101` (redefinição sem token)
- `app/lib/schemas/auth.ts:69-79` (nome com min 2 chars)

## Impacto
Compromete QUALQUER conta, inclusive ADMIN. Pivô para acesso total ao módulo financeiro e gestão de usuários.

## Sugestão de correção
1. Substituir o fluxo 2-step por envio de token: gerar UUID v4, persistir em `password_reset_token` com `expiresAt = now + 30min` + `usedAt`.
2. Enviar link com token por e-mail.
3. Step 2 exigir `token` válido e não usado.
4. Invalidar todas as sessões do membro após troca (`DELETE FROM Session WHERE membroId = ?`).
5. Rate-limit por IP e por e-mail (5/h).
6. Audit log com IP + UA.

## Critérios de aceite
- [ ] Não é possível redefinir senha sem token de uso único válido por ≤ 30min
- [ ] Token expirado/usedAt não permite redefinição
- [ ] Sessões anteriores do membro são invalidadas após reset
- [ ] Audit log registra reset com IP+UA
- [ ] Teste E2E: tentativa sem token retorna 422
- [ ] Rate-limit: 5 resets/h por e-mail + 10/h por IP

--- FIM ISSUE 1 ---
```

Labels: `security, critical`

```
--- ISSUE 2 ---

# [Segurança] Rotação imediata de SESSION_SECRET e remoção do histórico

**Labels:** `security`, `critical`

## Problema
`SESSION_SECRET="ba9585ca0efb2de1d6ca5c5b3b266ed6052077fbce73a2e6"` está COMMITADO em `.env.development` (git log confirma commit `176f3774 financeiro`). Quem tem acesso ao repo pode forjar cookies de sessão válidos para qualquer membro.

## Por que é explorável
`session.server.ts:37` usa o secret diretamente em `secrets: [process.env.SESSION_SECRET]`. O cookie assinado
```

```
por esse secret é aceito pelo app — `findUnique({where: {id: sid}})` no DB. Atacante pode escolher qualquer
UUID v4 como sid e o login será aceito.

## Evidência
- `.env.development:4` — secret literal
- `git log --diff-filter=A --name-only` retorna `.env.development` no commit `176f3774`

## Impacto
Forjamento de sessão universal. Atacante loga como qualquer membro sem precisar de credenciais.

## Sugestão de correção
1. **Rotacionar `SESSION_SECRET` em produção IMEDIATAMENTE** (`openssl rand -hex 32`).
2. **Remover `.env.development` do git history** (`git filter-repo --invert-paths --path .env.development`).
3. Adicionar `.env.development` em `.gitignore`.
4. Tratar secret antigo como comprometido — invalidar todas as sessões ativas (`DELETE FROM Session`).
5. Pre-commit hook (`gitleaks` ou check `SESSION_SECRET=`).

## Critérios de aceite
- [ ] Histórico do `.env.development` removido do repo
- [ ] Novo `SESSION_SECRET` em prod (≥ 32 chars, aleatório)
- [ ] Todas as sessões ativas invalidadas
- [ ] Pre-commit hook ativo
- [ ] `.env.development` no `.gitignore`

--- FIM ISSUE 2 ---
```

Labels: `security, critical`

```
--- ISSUE 3 ---

# [Segurança] Credenciais de ADMIN com senha fraca + chaves S3 reais em .env

**Labels:** `security`, `critical`

## Problema
Arquivo `.env` (filesystem dev) contém:
- `ADMIN_PASSWORD=senha123` (trivial, quebrada em <1ms)
- `STORAGE_ACCESS_KEY_ID` e `STORAGE_SECRET_ACCESS_KEY` reais (apontam para `coolify.esmirna.com.br:9000`)
- `STORAGE_TLS_REJECT_UNAUTHORIZED="false"` (MITM no canal S3)

Além disso, `prisma/seed.ts:25` tem fallback `?? 'admin123'` caso `ADMIN_PASSWORD` não esteja definida em env
— combinações perigosas se seed rodar em prod sem env.

## Por que é explorável
`STORAGE_PROVIDER=local` ignora as chaves S3 atualmente (`config.server.ts:91-98`), mas:
(a) `.env` com `senha123` é o primeiro vetor de força-bruta contra a conta `kingdevtec@gmail.com` se deploy
copiar o arquivo.
(b) Trocar `STORAGE_PROVIDER=garage` no deploy sem rotacionar as chaves reutiliza credenciais expostas.
(c) `rejectUnauthorized=false` aceita cert autoassinado/inválido.

## Evidência
- `.env:8-9` (`ADMIN_PASSWORD=senha123`)
- `.env:14-22` (chaves S3 + `rejectUnauthorized=false`)
- `prisma/seed.ts:25` (fallback `admin123`)

## Impacto
Credential stuffing contra ADMIN. MITM no canal S3. Seed de prod com senha trivial.

## Sugestão de correção
1. Trocar `ADMIN_PASSWORD` no `.env` por valor aleatório (≥ 12 chars, `openssl rand -base64 24`).
2. Rotacionar `STORAGE_*_KEY` no Garage/MinIO.
3. Setar `STORAGE_TLS_REJECT_UNAUTHORIZED=true` em prod.
4. Adicionar startup check que rejeita `rejectUnauthorized=false` em produção (exceto via `INSECURE_TLS=true`
opt-in documentado).
5. Remover fallback `admin123` do `seed.ts` — falhar com erro explícito se env não definida.

## Critérios de aceite
- [ ] `.env` sem `senha123` ou outras senhas de wordlist
- [ ] Chaves S3 rotacionadas
- [ ] `rejectUnauthorized=true` em prod
- [ ] Seed falha se `ADMIN_PASSWORD` não definida
```

```
- [ ] Teste: rodar seed sem env → erro

--- FIM ISSUE 3 ---
```

Labels: `security, high`

```
--- ISSUE 4 ---

# [Segurança] Bypass de rate-limit via campo clientIP no body do login

**Labels:** `security`, `high`

## Problema
`app/routes/public/login.tsx:56-61` aceita o IP do cliente a partir do próprio form (`formData.get(
'clientIP')`) quando os headers `x-forwarded-for` e `x-real-ip` estão ausentes. Atacante rotaciona `clientIP`
a cada POST para burlar o rate-limit.

## Por que é explorável
A lógica do IP é:
```ts
const ip =
  request.headers.get('x-forwarded-for')?.split(',')[0]?.trim() ??
  request.headers.get('x-real-ip') ??
  (formData.get('clientIP') as string) ??  // <- atacante controla
  'unknown';
```
Em qualquer deploy sem proxy reverso que preencha XFF, o atacante controla o IP usado pelo rate-limit e pelo
audit log.

Evidência
`app/routes/public/login.tsx:56-61`

Impacto
Bypass de rate-limit de login (3 falhas/hora). Força-bruta viável. Audit log poluído com IPs falsos.

Sugestão de correção
1. Remover leitura de `formData.get('clientIP')`.
2. Usar exclusivamente headers do proxy (XFF, X-Real-IP) + IP do socket TCP.
3. Em `NODE_ENV=production`, exigir pelo menos um header de proxy — caso contrário, rejeitar a request.

Critérios de aceite
- [ ] Sem leitura de `clientIP` do body
- [ ] Rate-limit não pode ser bypassado
- [ ] Audit log registra apenas IPs do proxy
- [ ] Teste: enviar POST com `clientIP` variável não afeta o bucket de rate-limit

--- FIM ISSUE 4 ---
```

Labels: `security, high`

```
--- ISSUE 5 ---

# [Segurança] KPIs de membros vazam totais globais para perfis com escopo restrito

**Labels:** `security`, `high`

## Problema
`app/routes/app/membros._index.tsx:84-91` usa `prisma.membro.count({where: baseWhere})` com `baseWhere = {}`.
DISCIPULADOR/LIDER_MINISTERIO logado vê totais GLOBAIS da igreja (total de membros, ativos, visitantes,
congregados) mesmo a listagem sendo escopada para apenas seus discípulos.

## Por que é explorável
`listMembros` aplica RBAC fina corretamente (`members.server.ts:149-154`), mas os 4 cards de KPI batem `count(
{where: {}})` direto, sem filtrar por `user.id` ou cargo. Quebra o padrão '404 (não vaza existência)' do RAG
`security-rbac-matrix`.

## Evidência
`app/routes/app/membros._index.tsx:84-91`

## Impacto
```

````
Vazamento de métricas globais (tamanho da igreja, nº de visitantes) para usuários com escopo restrito.

## Sugestão de correção
Construir `baseWhere` a partir de `user.cargo` (mesma lógica de `listMembros`):
```ts
const baseWhere: Prisma.MembroWhereInput =
  (user.cargo === 'DISCIPULADOR' || user.cargo === 'LIDER_MINISTERIO')
    ? { discipuladorId: user.id }
    : {};
```
Centralizar em `members.server.ts` como helper `getMembrosCountWhere(user)`.

## Critérios de aceite
- [ ] DISCIPULADOR logado vê KPI Total = nº de discípulos dele
- [ ] LIDER_MINISTERIO logado vê KPI Total = nº de discípulos dele
- [ ] ADMIN/PASTOR/SECRETARIO continuam vendo totais globais
- [ ] Teste cobrindo cada cargo

--- FIM ISSUE 5 ---
````

Labels: `security, medium`

```
--- ISSUE 6 ---

# [Segurança] Loader de edição de ministério expõe dados sem RBAC

**Labels:** `security`, `medium`

## Problema
`app/routes/app/ministerios.$id.tsx:65-91` (loader) NÃO chama `assertCanManageMinisterios`. Apenas a action valida `canEdit`. Qualquer usuário autenticado (inclusive DISCIPULADOR sem cargo) consegue abrir a página e ler dados internos (membros vinculados, funções, atividades, disponibilidades).

## Por que é explorável
O loader faz `findUnique` direto no Prisma sem checar permissão. DISCIPULADOR logado navega para `/app/ministerios/<qualquer-id>` e lê a lista nominal de membros do ministério, histórico de atividades e indisponibilidades.

## Evidência
`app/routes/app/ministerios.$id.tsx:65-91`

## Impacto
Vazamento de dados sensíveis de ministérios (membros nominais, funções, atividades) para usuários sem permissão.

## Sugestão de correção
Adicionar `assertCanManageMinisterios(user)` no loader (camada 2). Se a intenção for manter leitura ampla, criar matriz separada (`READ_MINISTERIO = ['ADMIN','PASTOR','SECRETARIO','LIDER_MINISTERIO']`).

## Critérios de aceite
- [ ] DISCIPULADOR recebe 403 ao abrir `/app/ministerios/:id`
- [ ] ADMIN/PASTOR/SECRETARIO/LIDER_MINISTERIO continuam lendo
- [ ] Teste cobrindo cada cargo

--- FIM ISSUE 6 ---
```

Labels: `security, high`

```
--- ISSUE 7 ---

# [Segurança] Senhas de demo + fallback admin123 em prisma/seed.ts

**Labels:** `security`, `high`

## Problema
`prisma/seed.ts` contém:
1. Fallback `process.env.ADMIN_PASSWORD ?? 'admin123'` (linha 25)
2. 5 senhas de demo hardcoded: `ricardo123`, `ana12345`, `marcos123`, `juliana123`, `bia12345` (linhas 79, 103, 127, 150, 228)
```

```
O check de produção (linha 283) só bloqueia se `ADMIN_*` estiver vazio, não se tiver valor fraco.

## Por que é explorável
Se alguém rodar `pnpm db:seed:prod` sem env definida, surge conta ADMIN com senha `admin123`. Senhas de demo
viram credenciais reais se seed rodar em prod acidentalmente.

## Evidência
`prisma/seed.ts:25, 79, 103, 127, 150, 228, 283`

## Impacto
Credenciais default em prod. Catálogo de senhas triviais.

## Sugestão de correção
1. Remover fallback `?? 'admin123'` — falhar com erro se env não definida.
2. Aceitar `DEMO_SEED_ALLOWED=true` em dev para criar usuários de demo; em prod, rejeitar a operação.
3. Mover contas de demo para `prisma/seed-demo.ts` separado.

## Critérios de aceite
- [ ] Falhar loud se `ADMIN_PASSWORD` não setada
- [ ] Contas de demo isoladas em script opcional
- [ ] `db:seed` em prod exige flag explícita

--- FIM ISSUE 7 ---
```

Labels: `security, medium`

```
--- ISSUE 8 ---

# [Segurança] Hardening de fallback de SESSION_SECRET e HMAC do storage local

**Labels:** `security`, `medium`

## Problema
Duas constantes hardcoded servem de fallback para `SESSION_SECRET`:
- `session.server.ts:37`: `secrets: [process.env.SESSION_SECRET ?? 'dev-only-not-secret']`
- `local-signed-url.server.ts:22`: `return 'dev-insecure-secret-change-me'`

Os checks em produção rejeitam env vazia, MAS qualquer NODE_ENV diferente de `production` aceita o fallback.
Atacante que conhece o repo pode forjar cookies de sessão E signed URLs do storage local contra qualquer
instância em `NODE_ENV=staging`, `NODE_ENV=test` ou sem `NODE_ENV` definido.

## Evidência
- `app/lib/session.server.ts:37`
- `app/lib/storage/local-signed-url.server.ts:22`

## Impacto
Forjamento de cookies e signed URLs se NODE_ENV ≠ production.

## Sugestão de correção
1. Throw explícito se SESSION_SECRET ausente em QUALQUER NODE_ENV (não só production).
2. Mesmo para o HMAC do storage local.
3. Para dev local, documentar no README: `openssl rand -hex 32 > .env.development`.

## Critérios de aceite
- [ ] Throw explícito se SESSION_SECRET ausente
- [ ] Throw explícito se HMAC secret ausente
- [ ] Teste: `delete process.env.SESSION_SECRET` → app não inicia

--- FIM ISSUE 8 ---
```

Labels: `security, low`

```
--- ISSUE 9 ---

# [Segurança] Endurecer logger de erros e adicionar lib de sanitização HTML

**Labels:** `security`, `low`

## Problema
1. `app/routes/api/uploads.ts:174` loga `errorDetails` completo (incluindo bucket, endpoint, key do S3) via
```

`console.error`. Em produção o response NÃO vaza, mas o log persiste.
2. Não há nenhuma lib de sanitização HTML no projeto (`dompurify`, `sanitize-html`). A única defesa XSS atual é o auto-escape do React + 1 escape manual em `membros.$id.tsx:249-274`.

Por que é explorável
1. Logs vazam infra interna via Docker logs / SIEM.
2. Renderizadores Markdown/HTML futuros adicionam risco XSS.

Evidência
- `app/routes/api/uploads.ts:174`
- `grep -rn 'dompurify\|sanitize-html' app/` retorna 0 hits

Impacto
Vazamento de infra via logs. Risco XSS futuro.

Sugestão de correção
1. Logar apenas `error.message` + correlation ID; manter detalhes em `Upload.rejectionDetails` (já persiste).
2. Adicionar `dompurify` + `@types/dompurify`.
3. Criar wrapper `safeHtml(input)` e aplicar em todos os pontos atuais e futuros.

Critérios de aceite
- [ ] Logs de produção não contêm bucket/endpoint/key
- [ ] Wrapper `safeHtml` usado em >= 1 ponto
- [ ] Teste com 5+ payloads XSS cobrindo o wrapper

--- FIM ISSUE 9 ---